

RoleRAG Interviewer

Deep Technical Project Explanation

A full walkthrough of the frontend, backend, RAG pipeline, database design, PDF reporting, and Netlify/Supabase/Groq deployment architecture.

1. Project Purpose

RoleRAG Interviewer is a full-stack AI interview screening system. It accepts a candidate resume, extracts a structured profile, retrieves role-specific knowledge from a RAG knowledge base, generates adaptive technical interview questions, evaluates the candidate's answers, stores the full interview trace, and lets the user download a PDF report.

The project is built to demonstrate more than a basic chatbot. It shows an end-to-end AI application architecture with:

- Resume parsing
- Role-aware RAG retrieval
- Adaptive question generation
- Turn-by-turn answer evaluation
- Persistent session storage
- PDF report generation
- Local development support
- Hosted Netlify + Supabase deployment support

The system currently has two execution modes:

- Local development mode: React + FastAPI + SQLite + ChromaDB + local sentence-transformer embeddings.
- Hosted deployment mode: React on Netlify + Netlify Functions + Supabase Postgres + pgvector + Supabase Storage + Groq chat + deterministic hash embeddings.

The two modes share the same product flow and API concepts, but use different infrastructure because local development can rely on local files and processes while Netlify requires serverless, externalized storage.

2. High-Level User Flow

From the user's point of view, the application works like this:

1. The user opens the RoleRAG Interviewer web app.
2. The user selects a target role, such as AI/ML Engineer.
3. The user uploads a PDF or TXT resume.
4. The frontend sends the role and resume to the backend.
5. The backend extracts resume text.
6. The backend parses the resume into a structured candidate profile.
7. The backend creates retrieval queries from the role, skills, projects, and experience.
8. The RAG layer searches the role-specific knowledge base.

9. The LLM layer generates the first technical interview question.
10. The frontend displays the question and lets the candidate answer.
11. The backend evaluates the answer and stores the score, strengths, gaps, and feedback.
12. If the interview is not finished, the backend generates the next adaptive question.
13. After the final turn, the backend generates a session summary.
14. The user can download a PDF report containing the interview details.

This creates a traceable interview loop:

```
Resume + Role -> Profile -> RAG Queries -> Retrieved Context -> Question
Question + Answer + Context -> Evaluation -> Next Question or Summary
Session Data -> PDF Report
```

3. Technology Stack

Frontend

- React: builds the single-page user interface.
- Vite: development server and production bundler.
- Axios: sends HTTP requests to the backend.
- CSS: custom styling for the interview interface.

Local Backend

- FastAPI: exposes REST API routes.
- SQLAlchemy: manages local database models and sessions.
- SQLite: stores interview sessions and turns locally.
- ChromaDB: stores and searches local RAG chunks.
- sentence-transformers: default local embedding provider.
- pypdf: extracts text from PDF resumes and KB documents.
- ReportLab: generates downloadable PDF reports.
- OpenAI SDK: optional LLM and embedding support.

Hosted Backend

- Netlify Functions: serverless API runtime.
- TypeScript: implementation language for the Netlify API.
- Supabase Postgres: hosted relational database.
- pgvector: vector storage and similarity search in Postgres.

- Supabase Storage: stores resumes, reports, and KB source files.
- Groq API: hosted LLM provider for chat-style tasks.
- OpenAI SDK: used as an OpenAI-compatible client for Groq.
- PDFKit: generates hosted PDF reports.
- pdf-parse: extracts text from uploaded PDFs in the serverless path.

Deployment and Build

- Netlify: hosts the frontend and serverless API.
- `netlify.toml`: defines build, publish, functions, and API redirects.
- Root `package.json`: installs Netlify/serverless dependencies and builds the frontend.
- Supabase SQL migration: creates the hosted database schema and vector search function.

4. Repository Structure

The project is organized around separate frontend, backend, hosted, and documentation areas:

```
RAG Interviewer/
├── backend/
│   ├── app/
│   │   ├── api/ # FastAPI routes
│   │   ├── core/ # configuration
│   │   ├── models/ # SQLAlchemy models
│   │   ├── schemas/ # request/response schemas
│   │   └── services/ # resume, RAG, LLM, report logic
│   └── data/
│       ├── chroma/ # local Chroma persistence
│       ├── kb_docs/ # local KB source files
│       ├── uploads/ # local resume uploads
│       └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/ # React UI components
│   │   └── services/ # Axios API client
│   ├── package.json
│   └── netlify/
│       ├── functions/
│       │   ├── api.ts # hosted API endpoint
│       │   └── _lib/ # hosted service helpers
│       └── supabase/
│           ├── migrations/ # hosted database schema
│           └── docs/ # architecture and project docs
├── netlify.toml # Netlify build and routing config
└── package.json # root hosted build dependencies
```

5. Frontend Architecture

The frontend lives in `frontend/src`. The root component is `App.jsx`.

`App.jsx` owns the main application state:

- `role`: selected target role.

- `file`: selected resume file.
- `session`: current interview session id.
- `profile`: parsed candidate profile returned by the backend.
- `question`: current interview question.
- `answer`: current answer typed by the user.
- `turns`: completed question-answer-evaluation records.
- `summary`: final session summary after the interview ends.
- `maxTurns`: configured interview length.
- `loading`: request state.
- `error`: user-visible error message.

The frontend is split into focused components:

- `CandidateEntry`: role selector, resume upload, and start button.
- `InterviewQuestion`: current question, answer textarea, context sources, submit action.
- `Stepper`: visual interview progress.
- `TurnLog`: profile, retrieved sources, and previous turns.
- `EvaluationCard`: score and evaluation display.
- `SummaryView`: final summary and PDF download action.
- `ErrorBanner`: displays API errors.

The frontend uses a small Axios client in `frontend/src/services/api.js`:

```
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '',
});
```

This design is important:

- In production, the frontend calls same-origin `/api/...` routes on Netlify.
- In local development, Vite proxies `/api` requests to the FastAPI backend.
- No hardcoded backend URL is required for deployment.

6. Frontend Request Flow

Starting an Interview

When the user clicks Start interview:

1. `App.jsx` creates a `FormData` object.
2. It appends `role`.

3. It appends the uploaded resume.
4. It sends POST `/api/interviews/start`.
5. The backend returns:
 - `session_id`
 - `parsed profile`
 - `first question`
 - `context_sources`
 - `max_turns`
6. The frontend saves these values in React state.

Submitting an Answer

When the user submits an answer:

1. The frontend sends POST `/api/interviews/{session_id}/answer`.
2. The request body contains the candidate's answer.
3. The backend evaluates the answer.
4. The backend either returns the next question or marks the interview complete.
5. The frontend adds the completed turn to `turns`.
6. If complete, the frontend fetches GET `/api/interviews/{session_id}` for the summary.

Downloading the PDF Report

The summary view links to:

`/api/interviews/{session_id}/report.pdf`

The backend generates a PDF response and returns it with:

Content-Type: `application/pdf`
Content-Disposition: `attachment`

That causes the browser to download the interview report.

7. Local FastAPI Backend

The local backend starts from `backend/app/main.py`.

Its responsibilities are:

- Create the FastAPI app.

- Enable CORS for local frontend origins.
- Initialize local database tables.
- Mount the interview API router.
- Expose health and root endpoints.

The main route module is `backend/app/api/interview.py`.

It defines these endpoints:

```
GET /health
POST /api/kb/ingest
POST /api/interviews/start
POST /api/interviews/{session_id}/answer
GET /api/interviews/{session_id}
GET /api/interviews/{session_id}/report.pdf
POST /api/interviews/{session_id}/reset
```

The local backend uses dependency injection for database sessions through SQLAlchemy. Each request gets a database session and closes it after the request completes.

8. Local Database Design

The local database models are in `backend/app/models/db.py`.

InterviewSession

Stores one interview session.

Important fields:

- `id`: primary key.
- `candidate_name`: extracted candidate name.
- `role`: selected target role.
- `resume_text`: extracted resume text.
- `extracted_profile`: structured JSON profile.
- `summary`: final interview summary.
- `created_at`: timestamp.

InterviewTurn

Stores each interview turn.

Important fields:

- `id`: primary key.

- `session_id`: foreign key to `InterviewSession`.
- `question`: generated interview question.
- `answer`: candidate answer.
- `retrieved_context`: RAG context used to generate the question.
- `evaluation`: structured evaluation JSON.
- `created_at`: timestamp.

The relationship is:

`InterviewSession 1 -> many InterviewTurn`

This is valuable because the app can explain how each question was produced and how each answer was evaluated.

9. Resume Processing

Resume logic lives in `backend/app/services/resume.py` for local mode and `netlify/functions/_lib/resume.ts` for hosted mode.

The resume pipeline has two steps:

Text Extraction

Supported file types:

- PDF
- TXT

For PDFs, the local backend uses `pypdf`. The hosted backend uses `pdf-parse`.

If the extracted text is too short, the backend rejects the upload because it cannot generate a reliable interview from an empty or unreadable resume.

Structured Profile Extraction

The parser extracts fields such as:

- `name`
- `emails`
- `phone numbers`
- `skills`
- `domains`
- `projects`

- project details
- experience highlights
- skill categories
- summary
- seniority signal

There are two parsing paths:

- LLM-assisted parsing when a chat model key is available.
- Rule-based fallback parsing when no LLM is available.

The fallback parser uses:

- regular expressions for emails, phone numbers, and years of experience
- skill keyword detection
- resume section detection
- project title and bullet extraction
- domain inference based on keywords

This fallback path makes the project more robust because the app can still run without paid LLM access.

10. RAG Knowledge Base Ingestion

The RAG ingestion endpoint is:

`POST /api/kb/ingest`

or for one role:

`POST /api/kb/ingest?role=AI/ML%20Engineer`

The project supports three role folders:

AI/ML Engineer -> AI_ML_Engineer
 Backend Engineer -> Backend_Engineer
 Data Science / Applied ML -> Data_Science_Applied_ML

The ingestion pipeline is:

1. Find documents for the selected role.
2. Read each PDF, TXT, or Markdown file.
3. Split the text into chunks.
4. Generate embeddings for each chunk.

5. Store chunks, metadata, and embeddings in the vector store.

The chunking strategy is word-based:

- chunk size: about 950 words
- overlap: about 160 words
- very small chunks are filtered out

The overlap helps preserve context between adjacent chunks. Without overlap, important definitions or examples could be split apart and retrieval quality could drop.

11. Local RAG with ChromaDB

In local mode, RAG is implemented in `backend/app/services/rag.py`.

The local vector store is ChromaDB, persisted under:

```
backend/data/chroma
```

Each role gets a collection name based on:

- role
- embedding provider
- embedding model

That prevents embedding-space conflicts. For example, OpenAI embeddings and local sentence-transformer embeddings should not be stored in the same collection because they have different vector spaces.

Local embedding configuration:

```
EMBEDDING_PROVIDER=local  
EMBEDDING_MODEL=all-MiniLM-L6-v2
```

OpenAI embeddings are still supported locally if configured:

```
EMBEDDING_PROVIDER=openai  
EMBEDDING_MODEL=text-embedding-3-small  
OPENAI_API_KEY=...
```

For development, local embeddings are the safer default because they do not require paid API quota.

12. Hosted RAG with Supabase and pgvector

In hosted mode, Netlify cannot rely on local ChromaDB persistence because serverless functions have ephemeral filesystems. The project therefore uses Supabase.

Hosted RAG data is stored in:

- Supabase Storage bucket kb-docs for original source files.
- Supabase Postgres table kb_chunks for chunks and embeddings.
- pgvector column embedding vector(1536) for vector search.

The SQL migration creates:

- interview_sessions
- interview_turns
- kb_chunks
- match_kb_chunks(...)
- pgvector extension
- pgcrypto extension
- HNSW vector index

The match_kb_chunks function receives:

- selected role
- query embedding
- match count

It returns the closest chunks by cosine distance:

```
1 - (kb_chunks.embedding <=> query_embedding) as similarity
```

The API calls this SQL function through Supabase RPC.

13. Hash Embeddings in Hosted Mode

The hosted project currently defaults to:

```
EMBEDDING_PROVIDER=hash  
EMBEDDING_MODEL=hash-embedding-1536
```

This was added because the project needed a free embedding path for deployment.

The hash embedding algorithm:

1. Tokenizes text into lowercase word-like tokens.
2. Hashes each token using SHA-256.
3. Maps the token to one of 1536 vector dimensions.

4. Adds either +1 or -1 depending on the hash.

5. Normalizes the vector length.

Benefits:

- Free to run.
- Deterministic.
- Works inside Netlify Functions.
- Produces 1536-dimensional vectors compatible with the Supabase schema.
- Avoids OpenAI quota failures.

Trade-off:

- It is not as semantically powerful as neural embeddings.
- It works best for keyword and lexical similarity.
- For production-quality semantic retrieval, a hosted neural embedding model or external vector database would be better.

This is a practical engineering trade-off: the hosted demo stays functional without paid embedding credits, while the code still supports real OpenAI embeddings if needed later.

14. Query Construction

Before retrieval, the backend builds multiple search queries from the resume profile.

Inputs include:

- selected role
- candidate skills
- projects
- domains
- experience level
- previous answer, for follow-up turns

Example query shapes:

```
AI/ML Engineer interview concepts for Python, RAG, FastAPI
practical debugging and design scenarios using Python, RAG
AI/ML Engineer concepts related to candidate project: RoleRAG Interviewer
AI/ML Engineer entry-level interview topics covering modeling, data pipelines, evaluation, experimentation, and
deployment
follow up question based on candidate answer: ...
```

This multi-query approach improves retrieval because one resume can contain several signals. Skills, projects, domains, and previous answers may each connect to different KB chunks.

15. Question Generation

Question generation is handled by:

- local: `backend/app/services/llm.py`
- hosted: `netlify/functions/_lib/llm.ts`

The system prompt asks the model to behave like a senior technical interviewer. It passes:

- role
- experience level
- extracted resume skills
- extracted projects
- profile highlights
- retrieved RAG context
- previous questions
- last answer, when available

The instruction is to generate exactly one interview question.

The generation logic is designed to avoid generic questions. It tries to connect the question to:

- candidate project details
- role-specific technical concepts
- retrieved KB context
- the previous answer
- production-style debugging or trade-off scenarios

If no LLM is available, fallback question generation still works. The fallback selects relevant projects and skills, then produces role-specific questions from templates.

16. Answer Evaluation

When the candidate submits an answer, the backend evaluates it against:

- the question
- the answer text

- retrieved context used for that turn

The evaluation output is structured JSON, typically including:

- score
- strengths
- gaps
- feedback
- follow-up suggestion

The local and hosted implementations both include fallback evaluation logic. That prevents the full flow from breaking when the LLM provider is unavailable.

This design makes the app reliable for demos:

- With an LLM: richer feedback.
- Without an LLM: deterministic scoring and feedback still appear.

17. Adaptive Interview Loop

The adaptive loop is the core product behavior.

For each answer:

1. Find the pending turn.
2. Save the candidate's answer.
3. Evaluate the answer.
4. Count completed turns.
5. If the interview is not finished:
 - build new queries using the last answer
 - retrieve fresh RAG context
 - generate the next question
 - save a new pending turn
6. If the interview is finished:
 - summarize the session
 - save the summary

This means the interview is not a static list of questions. Later questions can react to what the candidate said earlier.

18. PDF Report Feature

The PDF report feature was added so users can download the results.

Local mode uses:

```
backend/app/services/report.py
```

Hosted mode uses:

```
netlify/functions/_lib/report.ts
```

The report contains:

- candidate name or session id
- selected role
- interview date
- completed turn count
- parsed profile information
- each question
- each answer
- evaluation details
- final summary, when available

The report endpoint only allows download after at least one answer exists. This prevents empty reports.

In hosted mode, the generated report is also uploaded to the Supabase reports bucket and the report path is saved on the session row.

19. Hosted API Architecture

The hosted API endpoint is:

```
netlify/functions/api.ts
```

Netlify routes are configured in netlify.toml:

```
[[redirects]]
from = "/api/*"
to = "/.netlify/functions/api/:splat"
status = 200
```

This means:

```
/api/interviews/start
```

is served by:

```
/.netlify/functions/api/interviews/start
```

The function manually parses the path and dispatches to the correct handler.

This structure keeps deployment simple because there is one serverless API function instead of many small functions.

20. Supabase Data Model

The hosted schema is defined in:

```
supabase/migrations/20260610_netlify_interview_schema.sql
```

`interview_sessions`

Stores hosted sessions.

Fields include:

- `id`
- `role`
- `candidate_name`
- `resume_object_path`
- `report_object_path`
- `extracted_profile`
- `summary`
- `created_at`

`interview_turns`

Stores hosted turns.

Fields include:

- `id`
- `session_id`
- `question`
- `answer`
- `retrieved_context`
- `evaluation`
- `created_at`

kb_chunks

Stores hosted RAG chunks.

Fields include:

- id
- role
- source
- storage_path
- chunk_index
- content
- metadata
- embedding
- created_at

The unique key:

(role, source, chunk_index)

prevents duplicate chunk rows for the same document.

21. Supabase Storage Design

The hosted version uses three buckets:

resumes
reports
kb-docs

resumes

Stores candidate resume uploads.

reports

Stores generated PDF reports.

kb-docs

Stores source knowledge-base files by role folder:

kb-docs/
■■■ AI_ML_Engineer/

■■■ sample_notes.txt

The ingestion endpoint reads from this bucket, chunks the documents, embeds them, and writes rows into kb_chunks.

22. Groq Integration

The hosted backend uses Groq for chat-style LLM tasks.

The code uses the OpenAI SDK because Groq exposes an OpenAI-compatible API endpoint:

`https://api.groq.com/openai/v1`

The hosted config supports:

GROQ_API_KEY
GROQ_MODEL

The default model in the example configuration is:

`llama-3.1-8b-instant`

The code also supports real OpenAI chat if an OpenAI key is provided instead, but Groq is the current deployment-friendly path.

23. Environment Configuration

Local backend configuration lives in:

`backend/.env`

Hosted example configuration lives in:

`.env.netlify.example`

Important hosted variables:

SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_RESUME_BUCKET
SUPABASE_REPORT_BUCKET
SUPABASE_KB_BUCKET
GROQ_API_KEY
GROQ_MODEL
EMBEDDING_PROVIDER
EMBEDDING_MODEL
MAX_TURNS

The service role key must never be exposed to the browser. It is only used inside Netlify Functions.

The project also includes Netlify secret scanning omissions for non-secret values like bucket names and model names. This prevents harmless config strings from blocking deployment.

24. Error Handling

The project handles common errors at several layers.

Frontend:

- Displays API errors through `ErrorBanner`.
- Tracks loading state.
- Prevents empty flows from silently failing.

Local backend:

- Rejects unsupported file types.
- Rejects resumes with too little extracted text.
- Converts embedding configuration failures into 503 responses.
- Returns 404 for missing sessions.
- Returns 400 for too-short answers or missing pending questions.

Hosted backend:

- Uses `HttpError` for controlled failures.
- Uses `errorResponse` to convert thrown errors into API responses.
- Lazily initializes Supabase so missing env vars create useful errors instead of raw startup crashes.
- Catches RAG retrieval failures in some places so the interview can still continue with no context.

25. Security Notes

Important security practices in this project:

- `.env` files should not be committed.
- API keys should be stored in Netlify environment variables.
- Supabase service role key should only exist server-side.
- Resume and report buckets should be private.
- The frontend should never receive the service role key.
- Keys accidentally pasted or exposed during development should be rotated.

The hosted architecture intentionally keeps secrets inside Netlify Functions. The browser calls `/api`, and the function performs privileged Supabase operations.

26. Deployment Flow

The Netlify deployment flow is:

1. Push code to GitHub.
2. Connect the GitHub repository to Netlify.
3. Netlify runs the root build command:

```
npm run build
```

4. The root build command installs frontend dependencies and builds the Vite app:

```
npm --prefix frontend ci && npm --prefix frontend run build
```

5. Netlify publishes:

```
frontend/dist
```

6. Netlify deploys functions from:

```
netlify/functions
```

7. Requests to `/api/*` are rewritten to the Netlify function.
8. Supabase stores sessions, turns, KB chunks, resumes, and reports.

27. Local Run Flow

For local development, run the backend:

```
cd backend
source venv/bin/activate
python -m uvicorn app.main:app --reload
```

Then run the frontend:

```
cd frontend
npm run dev
```

The frontend opens on:

```
http://localhost:5173
```

The backend runs on:

`http://localhost:8000`

Vite proxies `/api` to the backend, so the frontend code stays the same as production.

28. Knowledge Base Validation

For local mode:

1. Put KB documents under `backend/data/kb_docs/<role-folder>/`.
2. Start the FastAPI backend.
3. Run:

```
curl -X POST "http://localhost:8000/api/kb/ingest?role=AI/ML%20Engineer"
```

For hosted mode:

1. Upload KB documents to Supabase Storage bucket `kb-docs`.
2. Use role folders such as `AI_ML_Engineer`.
3. Redeploy or ensure Netlify env vars are active.
4. Run:

```
curl -X POST "https://YOUR_SITE.netlify.app/api/kb/ingest?role=AI/ML%20Engineer"
```

The response should include:

```
{
  "chunks_ingested": 1,
  "roles_scanned": ["AI/ML Engineer"]
}
```

The exact chunk count depends on document length.

29. Important Engineering Trade-Offs

Local vs Hosted Backend

The local backend is ideal for development because it is simple and fast:

- SQLite is easy to run locally.
- ChromaDB can persist to disk.
- local embeddings avoid paid API quota.

The hosted backend is required for Netlify because:

- Netlify Functions are stateless.
- Local SQLite files are not reliable in serverless deployment.
- Local Chroma persistence is not suitable for serverless.
- Uploaded files need external storage.

Hash Embeddings vs Neural Embeddings

Hash embeddings are free and deployment-friendly, but less semantically rich.

Neural embeddings are better for production semantic retrieval, but require a paid or hosted embedding provider.

The project keeps both paths available.

Fallback Logic

The fallback logic makes the project demo-safe. Even if the LLM provider fails, the app can still:

- parse resumes approximately
- generate reasonable questions
- evaluate answers
- summarize sessions

That is important because external AI APIs can fail due to quota, latency, or configuration issues.

30. What Happens Internally From Start to Finish

This is the complete technical lifecycle for one interview:

1. User uploads a resume and chooses a role.
2. React stores the selected role and file in component state.
3. React sends multipart form data to `/api/interviews/start`.
4. The backend validates the role and file.
5. The backend extracts resume text.
6. The backend rejects the resume if extracted text is too short.
7. The backend parses the text into a structured profile.
8. The backend stores the resume and session metadata.
9. The backend builds role-aware search queries from the profile.
10. The RAG layer embeds those queries.
11. The vector store returns the closest KB chunks for that role.

12. The LLM layer receives resume profile, role, context, and previous questions.
13. The LLM or fallback generator creates one question.
14. The backend stores the first turn with question and retrieved context.
15. The frontend displays the question.
16. The candidate writes an answer.
17. React sends the answer to `/api/interviews/{id}/answer`.
18. The backend finds the pending unanswered turn.
19. The backend evaluates the answer.
20. The backend stores the answer and evaluation.
21. If more turns are allowed, the backend builds follow-up retrieval queries using the last answer.
22. The RAG layer retrieves fresh context.
23. The LLM layer generates the next adaptive question.
24. The backend stores a new pending turn.
25. The frontend displays the new question.
26. Steps 16 to 25 repeat until `MAX_TURNS` is reached.
27. The backend summarizes the full session.
28. The frontend fetches and displays the final summary.
29. The user clicks download report.
30. The backend generates a PDF from session and turn data.
31. The browser downloads the PDF report.

31. Why This Project Is Strong

The project demonstrates practical AI application engineering because it includes:

- A real frontend and backend.
- Resume-aware personalization.
- Retrieval-augmented generation.
- Adaptive multi-turn behavior.
- Persistent audit trail.
- PDF report generation.
- Local and hosted architecture.
- External database and storage migration.
- Serverless deployment considerations.
- Fallback behavior for API failures.

It is not only a prompt demo. It is a complete AI workflow with state, retrieval, evaluation, persistence, and deployment.

32. Future Improvements

Strong next improvements would be:

- Add user authentication.
- Add admin UI for uploading KB files.
- Replace hash embeddings with a production embedding provider.
- Add automatic KB ingestion after upload.
- Add streaming responses for better UX.
- Add interview templates per role.
- Add scoring rubrics per topic.
- Add analytics dashboard.
- Add automated tests for API routes.
- Add background jobs for large KB ingestion.
- Add signed URLs for private report downloads.

33. Summary

RoleRAG Interviewer is an end-to-end resume-aware technical interview platform. The frontend provides the candidate experience, the backend orchestrates resume parsing, retrieval, question generation, evaluation, and reporting, and the deployment scaffold moves the system onto Netlify and Supabase for hosting.

The most important technical idea is traceability. Every generated question is connected to the candidate profile and retrieved context, every answer receives structured feedback, and every session can be reviewed later through stored turns and a downloadable PDF report.

That makes the project useful as a practical demonstration of full-stack AI engineering, not just isolated LLM prompting.